Embeddings and position

From IDs to vectors

LLM chapter, after LLM-2: the box opens from the input side. Every GPT-2 (2019) number on these slides is read from its real weights.

Before we start: two videos and a demo

Recommended viewing

3Blue1Brown, Transformers, the tech behind LLMs | Deep Learning Chapter 5 (2024). From 12:27: word embeddings as directions in space.

Jia-Bin Huang, How Rotary Position Embedding Supercharges Modern LLMs [RoPE] (2024, 14 min). Sinusoids, then RoPE, then stretching it: today's second half, in the same order.

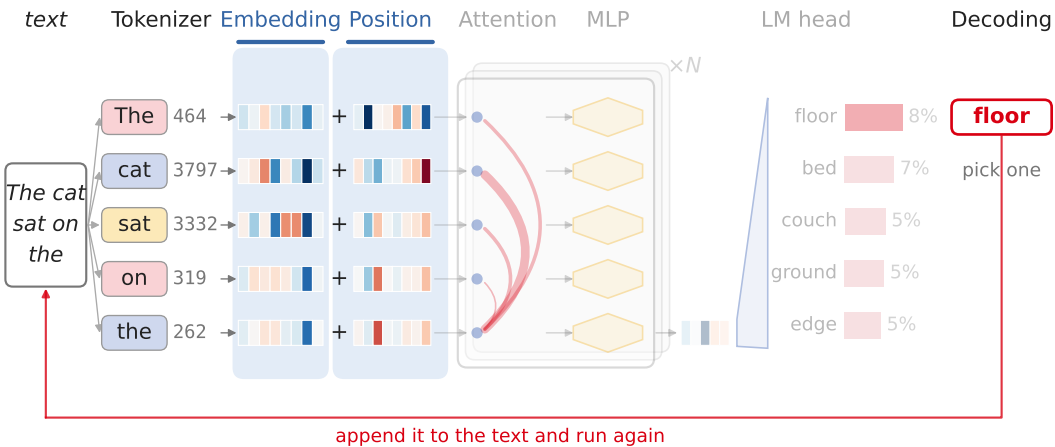
Live demo: projector.tensorflow.org

10,000 word vectors (word2vec, 2013; 200 numbers each) in the browser: search a word, list its nearest neighbours, squash all 10,000 into 3-D with PCA (ch10), t-SNE (2008) or UMAP (2018).

Try:

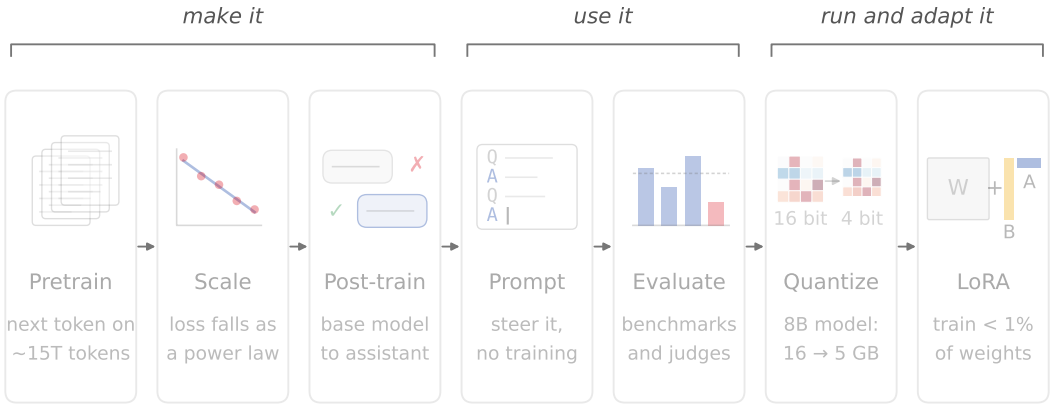
- a weekday, a city, a royal title
- an adjective: is its opposite among the neighbours?
- switch PCA to UMAP and watch clusters form

Where we are: one forward pass



LLM-2 did the last box. Today: an ID becomes a vector, and the vector learns where it sits.

Where we are: the life of a model



Later: where the weights come from, and what you do with them once you have them.

Same words, opposite news

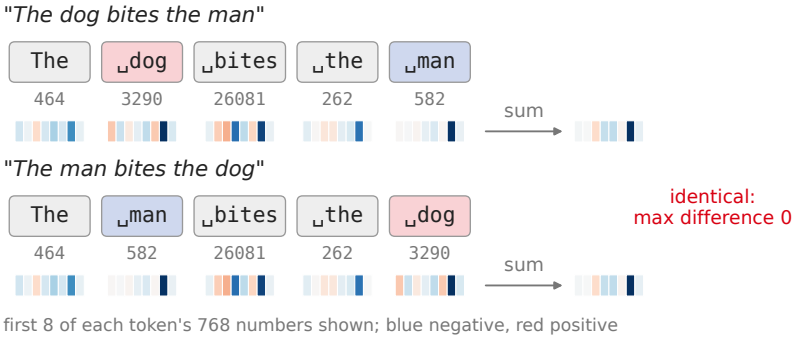
"The dog bites the man"
"The man bites the dog"

Predict: the model's first step looks every token up in a table and gets back a list of numbers. Are the two sentences still different after that step?

ուրա պարավը, բերեք ատամը քաշեմ

karginhaghordum.am/sketch/3bEwVKXicsQ

Same five numbers, same five vectors



first 8 of each token's 768 numbers shown; blue negative, red positive

The same five token IDs in a different order. Each ID fetches its row of the table, so both sentences arrive as the **same five vectors**. Add them up and the totals agree to the last bit.

Aside: without "The", "dog bites man" and "man bites dog" are not even the same tokens. The first word has no space before it, and "dog" is a different token from "␣dog" (LLM-1).

Outline

An ID becomes a vector

Where is the order?

Option 1: learn the positions

Option 2: write the positions down

Option 3: rotate, do not add

Longer and longer

An ID becomes a vector

3290 is a row number, not a quantity.

An ID is a name, not a number

- LLM-1 ended with integers: " cat" is **3797**, " dog" is **3290**. LLM-2 and the API notebook jumped from there straight to probabilities. Today: the first step in between.
- Is a cat 507 "more" than a dog? Is " dog" + 1 a puppy? The IDs were handed out by the order of BPE's merges (Sennrich et al., 2016; LLM-1). Their sizes mean nothing.
- [53] hit the same wall with one-hot vectors: every word equally far from every other, so "cat" is no closer to "dog" than to "Tuesday".

What we want: one list of numbers per token, where tokens used in similar ways get similar numbers. And we want the model to **learn** those numbers, not us to choose them.

By hand: a one-hot vector times a matrix picks a row

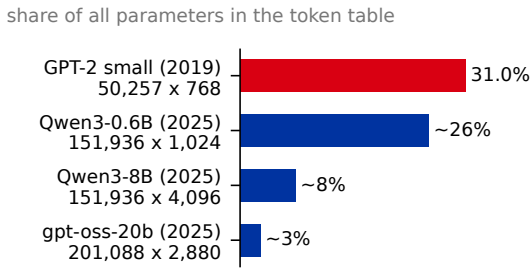
A 4-token vocabulary (the, cat, sat, on), 3 numbers per token. The table E has one row per token. "cat" is token 1, so its one-hot vector has a 1 in place 1:

$$\underbrace{\begin{pmatrix}0 & 1 & 0 & 0\end{pmatrix}}_{\text{one-hot(cat)}} \underbrace{\begin{pmatrix}0.2 & -0.1 & 0.5\\ 0.9 & 0.3 & -0.4\\ -0.6 & 0.8 & 0.1\\ 0.0 & -0.5 & 0.7\end{pmatrix}}_{E\ (4\ \text{tokens}\times 3\ \text{numbers})} = 0\cdot\text{row}_0+1\cdot\text{row}_1+0\cdot\text{row}_2+0\cdot\text{row}_3 = \begin{pmatrix}0.9 & 0.3 & -0.4\end{pmatrix}$$

- Every zero wipes out a row; the single 1 keeps row 1. So the "embedding layer" is a matrix multiplication that is really a **table lookup**: code just reads row 1.
- Training works the same way: the gradient reaches only the rows that were looked up. A token that never shows up as input gets nothing here. (With tying, a few slides on, the output side adds a small push to every row.)

GPT-2's table

- One row per token: **50,257** rows.
- 768 numbers per row.
- $50{,}257 \times 768 = 38{,}597{,}376$ numbers: **31.0%** of GPT-2 small's 124M parameters (LLM-1's vocabulary trade-off).



Bigger models spend a smaller share on it: the table grows with the vocabulary and the width, the rest of the model grows much faster.

Qwen3 and gpt-oss: vocabulary and width from their config.json, totals from their model cards (rounded there, hence "~"), checked 4 Oct 2026.

Who fills the table?

- At the start: small random numbers. "cat" and "dog" are as unrelated as any two rows.
- Then the table trains with the rest of the model, by backpropagation, on one task: predict the next token. Nobody ever tells it what a cat is.
- You have built one already: the ch11 name inventor was **an embedding table plus an MLP**. Its table had 40 rows (38 letters, plus the start and end symbols). Same idea, now with 50,257 rows.

If two tokens keep appearing in the same places, the gradient keeps pushing their rows in the same direction. Similar use $\Rightarrow$ similar rows. Two pictures: what the table looks like before and after, and why the pushes line up.

Random at the start, organised at the end

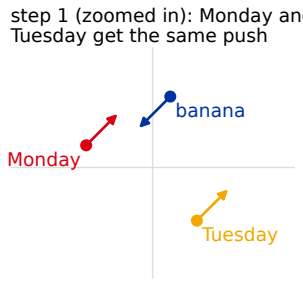
random start		after training	
␣coffee		␣coffee	
␣Chicken	0.17	␣espresso	0.67
sha	0.15	␣tea	0.62
Bir	0.14	␣caffeine	0.62

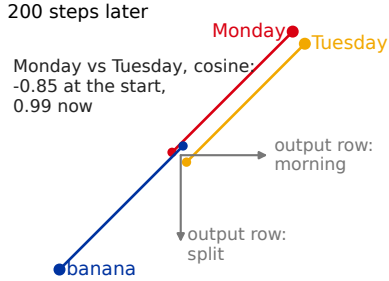
random start		after training	
␣seven		␣seven	
␣respectfully	0.14	␣six	0.87
␣Person	0.14	␣eight	0.86
␣silicone	0.14	␣five	0.85

- Left of each pair: a table made the way GPT-2's started, every number drawn at random (a normal distribution with spread 0.02). The nearest rows are junk, all near 0.15: the best of 50,257 chance matches.
- Right: the same two words in GPT-2's trained table. Drinks next to coffee, numbers next to seven.

Same words, same 768 numbers per row. The only difference is training on "predict the next token".

Why the pushes line up: a toy





Toy, 2 numbers per row: Monday and Tuesday are always followed by "morning", banana by "split". Each step nudges a row to raise its next word's probability (softmax over the output rows, LLM-2): toward morning's output row and away from split's, for both days alike. banana gets the opposite push.

Same next words ⇒ same nudges ⇒ similar rows. The toy holds the output rows still; in GPT-2 they train too, and they are the same table (tying, a few slides on).

Which rows of GPT-2's table sit closest to " Monday"?

Closest = largest cosine similarity (the angle between two rows).

Why the angle and not the plain dot product: GPT-2's rows range from 2.45 to 6.32 in length, and a dot product rewards length as well as direction.

Commit to an answer.

What the table learned

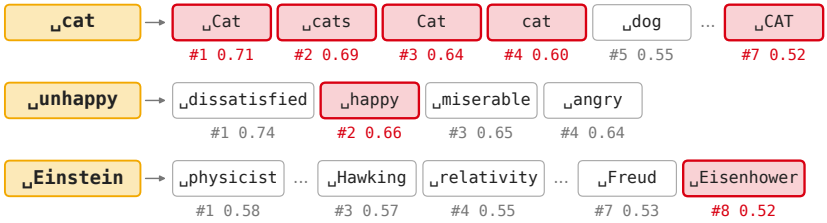
␣Monday		␣1999		␣Harry		␣Batman	
Tuesday	0.89	1998	0.89	Hermione	0.63	Superman	0.68
Thursday	0.85	1997	0.86	Voldemort	0.61	Gotham	0.62
Wednesday	0.84	1995	0.84	Ginny	0.59	superhero	0.60

␣Messi		␣lol		␣Armenia		␣vodka	
Ronaldo	0.66	haha	0.79	Armenian	0.76	whiskey	0.70
Guardiola	0.61	:)	0.64	Azerbaijan	0.72	booze	0.68
Juventus	0.60	;)	0.62	Kazakhstan	0.60	bourbon	0.66

The three nearest rows of each word, with their cosine similarity. Days next to days, years next to years, Hermione next to Harry, Ronaldo next to Messi. Spelling variants of the word itself are skipped here (next slide).

Nobody labelled a single token. All of this came from predicting the next one.

Neighbours share contexts, not meanings



- " cat": the nearest rows are the same word spelled differently. LLM-1's tokenizer gave them separate IDs; training pulled their rows back together.
- " unhappy": its second neighbour is its **opposite**. "I was ___ with the result" takes both. Opposites live in the same places.
- " Einstein": physicists, then at #8 " Eisenhower": another famous 20th-century name, with similar letters. Not physics.

A row records how a token is **used**. Usually that tracks meaning. Not always. Linguists said it in 1957: "You shall know a word by the company it keeps" (Firth).

The glitch tokens found each other



- LLM-1's glitch token's nearest rows are **other glitch tokens**: Reddit user names and scraped-page leftovers.
- The 20 rows closest to the table's average row: these three, plus 17 single-byte tokens.
- Why: these tokens were in the tokenizer's training text and almost never in the model's. Their rows got no gradient of their own, only the push every absent token gets at the output, where the same table scores the next token (tying, a few slides on): "not this one". That push points the same way for all of them, so they drifted together, toward the table's average.

Land & Bartolo (2024), "Fishing for Magikarp", EMNLP: thousands of such under-trained tokens in today's open models, found from the weights alone.

Directions carry meaning (mostly)

Take row *b*, subtract row *a*, add row *c*, and ask which row is nearest:

query	nearest row	cosine
king − man + woman	**queen**	0.69
actor − man + woman	**actress**	0.82
cats − cat + mouse	**mice** (an irregular plural)	0.70
German − Germany + Armenia	**Armenian**	0.79
better − good + bad	**worse**	0.63
Paris − France + Italy	"Paris" (no space), Milan; Rome only 5th	0.62
walked − walk + swim	swimming, not swam	0.61

Fine print: the three input words are removed from the answers, as everyone does (Mikolov et al., 2013). Without that rule, 3 of the 5 green rows answer with one of their own inputs: king − man + woman is nearest to **king** (Nissim et al., 2020, make this point). So: a real tendency of the table, not a law.

Analogies find stereotypes, and the rule inflates them

- Bolukbasi et al. (2016) asked word2vec (2013), trained on Google News: man is to computer programmer as woman is to ...? Answer: **homemaker**. Crowd workers rated 29 of 150 analogies the vectors produced as stereotypes.
- The table learned from text people wrote, so it carries their associations. Their fix: find a "gender direction" and remove it from rows that should not have one.
- Nissim, van Noord & van der Goot (2020) pointed at the rule from the last slide: an input word may never be the answer. Lift it, and the answer is **computer programmer** itself. Their title: "Man is to Doctor as Woman is to **Doctor**".

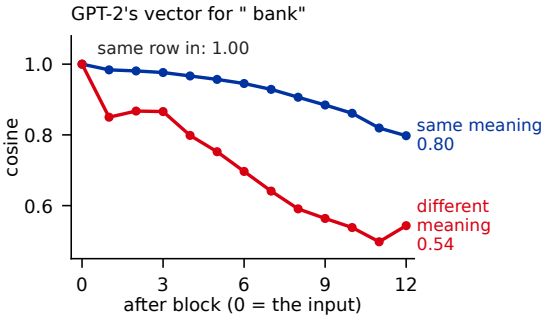
Both are true: embeddings carry the biases of their text, and the analogy trick exaggerates them. One striking example is not a measurement.

One row per token, whatever the sentence

- "We sat on the river **bank**" and "I opened a **bank** account": both "bank"s are token 3331, so both get **the same row**.
- The table cannot see the sentence. It answers "which token is this?", never "what does it mean here?".
- Telling the two banks apart is the job of the next layers, which mix each vector with its neighbours (LLM-4).

The table gives each token a **starting point**. The vectors that come out of the last layer are different for every occurrence of a token; the ones that go in are not.

Measured: same row in, different vectors out



12 sentences of 6 tokens, " bank" last: 6 river ("He sat on the river bank") and 6 money ("He paid cash at the bank"). Same token, same slot: all 12 go in as the same vector.

After each of GPT-2's 12 blocks, compare " bank"'s vectors. Blue: two of the same meaning. Red: river vs money.

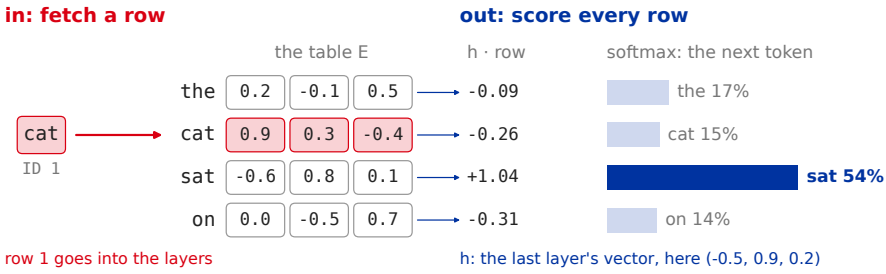
By the last block the meanings have drifted apart: **0.80** within a meaning, **0.54** across. GPT-2 reads left to right, so " bank" only sees "river" or "cash" before it.

Fine print: all vectors of a layer share one big common part (river vs money scores 0.99 after the last block as is), so each layer's average over an ordinary paragraph is subtracted first.

`py_src/embeddings_position_figs.py`, `fig_bank_layers`.

The same table, used twice

- At the very end, the model must turn its last vector h back into a token. It scores h against **every row of the same table** (one dot product per row). Those 50,257 scores are the logits LLM-2 started from: softmax, temperature and top-p all act on them.
- One table, two jobs: reading tokens in and writing them out. This is called **tying** the input and output embeddings. GPT-2 does it with its 50,257 rows.



The toy table from the lookup slide. "sat" wins: its row gives the largest dot product with h, $(-0.5)(-0.6) + (0.9)(0.8) + (0.2)(0.1) = 1.04$.

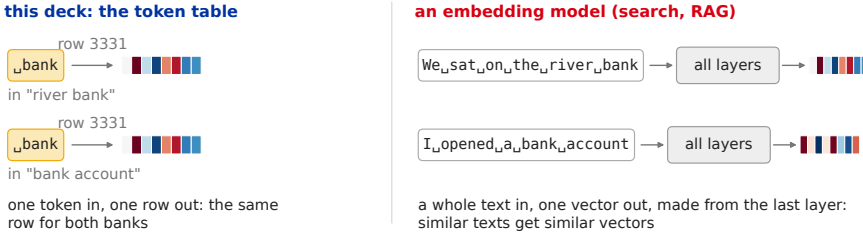
Who ties, and why

- The output step is often called the **unembedding**, or the language-model head (LM head, the map's name for it).
- Small models tie, big ones usually keep two tables, one for each job. Tied: Qwen3-0.6B (2025), Gemma 4 E2B (2026). Not tied: Qwen3-8B (2025), gpt-oss-20b (2025), DeepSeek-V3 (2024).

Sharing works because both jobs want the same thing: tokens used alike get vectors alike. Why the split: in a small model the table is a quarter or more of all parameters (the GPT-2 slide), so sharing it saves a lot. In a big one it is a few percent.

`tie_word_embeddings` in each model's config.json, checked 4 Oct 2026.

Not this deck: embedding models



- Search and retrieval-augmented generation (RAG, 2020; later in the course) use another "embedding": **one vector per whole text**, from a transformer's last layer, trained so that texts that mean the same land close.
- OpenAI's text-embedding-3 (2024): 1,536 or 3,072 numbers, \$0.02 or \$0.13 per million tokens. Open: Qwen3-Embedding-8B (2025), up to 4,096 numbers, top of the multilingual Massive Text Embedding Benchmark (MTEB, 2022) in June 2025.

Same word, two things: the token table is a model's **first** step, an embedding model's vector comes out of its **last**.

OpenAI's model pages and embeddings guide; Qwen3-Embedding-8B model card; checked 6 Nov 2026.

Where is the order?

A table lookup has no idea where the token sat.

Back to the dog

- " dog" is the 2nd token of "The dog bites the man" and the 5th of "The man bites the dog". The table hands back the same row both times.
- So after the lookup, each sentence is just **five vectors**, and they are the same five. The model has received a **set**: which vectors, but not in what order.
- "Dog bites man" is not news. "Man bites dog" is. The difference is entirely in the order.

Writing the vectors in a list on the slide gives them an order. The question is whether anything **inside the model** can tell which one was first.

Why the next layer cannot recover it

A preview of LLM-4. The next layer updates each vector like this:

1. compare it with every other vector: a **dot product** of their numbers;
2. turn those scores into weights;
3. add the vectors up with those weights.

Each step uses only **what is in** the vectors. None of them reads a slot number, and a weighted sum gives the same total whatever order you add in. Next slide: by hand. The attention lecture proves it in general: shuffle the inputs and every output comes back unchanged, just shuffled.

An RNN never had this problem: it read tokens one at a time, so order was built into the computation ([53]). Attention reads all tokens at once - that is why it is fast - and the price is that order is gone.

By hand: shuffle the sentence, and nothing changes

Toy vectors with 2 numbers: dog $= (1, 0)$, bites $= (1, 1)$, man $= (0, 1)$. The next layer, simplified (LLM-4 adds learned matrices), updates "dog":

	dog bites man			**man bites dog**		
slot	vector	score	weight	vector	score	weight
0	dog $(1, 0)$	1	0.42	man $(0, 1)$	0	0.16
1	bites $(1, 1)$	1	0.42	bites $(1, 1)$	1	0.42
2	man $(0, 1)$	0	0.16	dog $(1, 0)$	1	0.42

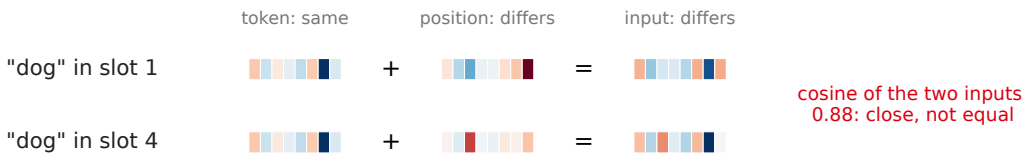
1. Score: the dot product with dog's vector. dog $\cdot$ bites $= 1 \cdot 1 + 0 \cdot 1 = 1$.
2. Weight: softmax of the scores, as in multiclass classification: e^1, e^1, e^0 divided by their sum gives $0.42, 0.42, 0.16$.
3. Output: $0.42 \cdot (1, 0) + 0.42 \cdot (1, 1) + 0.16 \cdot (0, 1) = (\mathbf{0.84},\ \mathbf{0.58})$ - in **both** sentences.

Each weight travels with its vector, not with its slot. Shuffling the sentence only shuffles the rows of this table.

The fix: write the slot into the vector

Give every **slot** its own vector too, and add it to the token's row (slots count from 0, as in code: the first token sits in slot 0):

$$\text{input}_i = \underbrace{E[\text{token}_i]}_{\text{which token}} + \underbrace{P[i]}_{\text{which slot}}$$



Now "dog in slot 1" and "dog in slot 4" are different inputs, and anything that compares vectors can tell them apart. Real GPT-2 rows above: the first 8 of 768 numbers.

By hand: add positions, and dog's output changes

Same toy, plus a vector per slot: slot 0 $= (0.5, 0)$, slot 1 $= (0, 0)$, slot 2 $= (0, 0.5)$. Add them, then run the same three steps for "dog":

	dog bites man			**man bites dog**		
slot	input	score	weight	input	score	weight
0	dog $(1.5, 0)$	2.25	0.63	man $(0.5, 1)$	1.00	0.25
1	bites $(1, 1)$	1.50	0.30	bites $(1, 1)$	1.50	0.42
2	man $(0, 1.5)$	0	0.07	dog $(1, 0.5)$	1.25	0.33
	dog's output: $(\mathbf{1.25}, \mathbf{0.40})$			dog's output: $(\mathbf{0.87}, \mathbf{0.84})$		

"dog" is now a different vector in each sentence ($(1.5, 0)$ in slot 0, $(1, 0.5)$ in slot 2), so every score changes, so the weights and the output change.

Same words, different outputs: the layer can now tell who bit whom. Yet the three inputs still add up to $(2.5, 2.5)$ in both sentences. That is the trap on the next slide.

Trap: adding positions does not fix a sum

Take the cold open again, with position vectors added this time, and add up the five inputs:

$$\sum_i \big(E[\text{token}_i] + P[i]\big) \;=\; \sum_i E[\text{token}_i] \;+\; \sum_i P[i]$$

Both sentences have the same tokens and the same five slots, so both sums are the same. We measured it on GPT-2: the two totals still agree to the last bit. The toy said the same: $(2.5, 2.5)$ twice.

Positions do not change the **total**; they change **each vector**. That is enough, because the next layer compares vectors pair by pair before it adds anything up (last slide: dog's output moved from $(1.25, 0.40)$ to $(0.87, 0.84)$). A model that only summed or averaged its inputs would stay blind to order.

Option 1: learn the positions

Another table, one row per slot.

GPT-2's position table

- A second table, P: one row per slot, 768 numbers per row, trained by backpropagation exactly like the token table.
- It has **1,024** rows: 786,432 numbers, 0.63% of GPT-2. Cheap.
- Rows 0 to 1,023: slot 1,024 has no row. GPT-2 cannot read a 1,025th token at all: the limit is built into the shape of a matrix. This limit is the model's **context length**: the most tokens it can read at once.
- BERT (2018) did the same with 512 rows. Longer documents had to be cut into pieces.

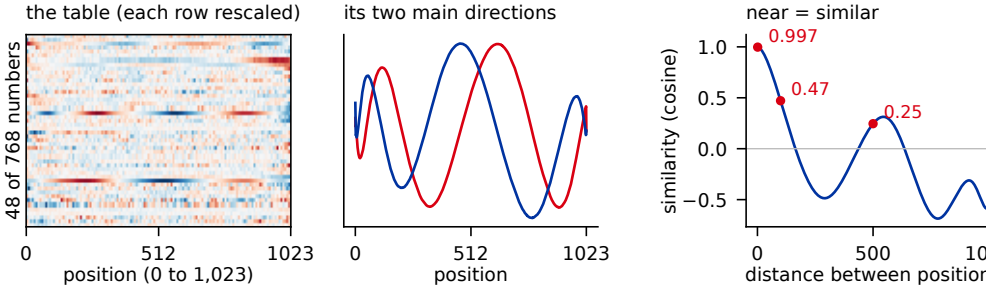
Nobody told GPT-2 that slot 7 comes between slots 6 and 8. To the training procedure they are 1,024 unrelated rows.

After training, what do those 1,024 rows look like?

Random noise? Something smooth? Something else?

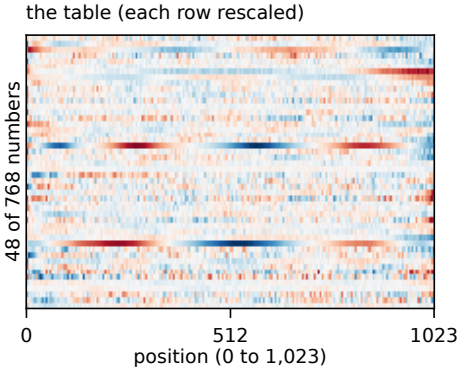
Commit to an answer.

It learned waves on its own



Three views of the same 1,024 rows: the raw numbers (left), a two-number summary of each row (middle), and how alike two rows are by distance (right). One at a time, next.

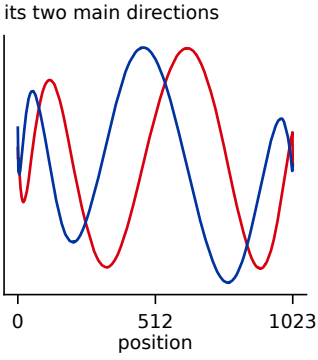
View 1: the table itself



- Each **column** is one slot: its row of numbers, stood on end. Slot 0 on the left, slot 1,023 on the right.
- Only 48 of the 768 numbers fit. Red is positive, blue negative. Each number is stretched to its own range, so the faint ones show too.
- Read one **horizontal line**: the same number, slot after slot. It rises and falls slowly, in long bands.

Neighbouring columns look almost the same. The rows started as random noise, and nothing in training said that slot 7 sits between 6 and 8. Training made them smooth anyway.

View 2: two numbers summarise each row



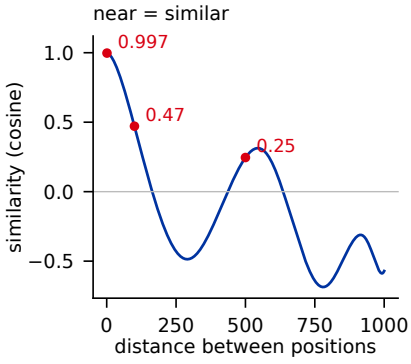
- 768 numbers per row are too many to plot. PCA (ch10) finds the few **directions** in which the 1,024 rows differ most.
- Three directions hold 90% of the differences, so each row is roughly

$$\text{average row} + a \cdot \text{dir}_1 + b \cdot \text{dir}_2 + c \cdot \text{dir}_3.$$

- The plot shows a (red) and b (blue) for every slot: how much of direction 1 and of direction 2 that slot's row contains.

Both are slow waves, a few swings across the 1,024 slots, with blue running about a quarter-swing ahead of red. That is what a turning clock hand looks like from two sides, a sine and a cosine. Option 2 builds exactly this on purpose.

View 3: how alike are two slots, by distance?



Pick two slots k apart, take the cosine of their two rows (as they are, nothing subtracted), and average over every such pair of slots. Plot that against k.

- $k = 1$: **0.997**. Neighbours are almost the same row.
- $k = 100$: 0.47. $k = 500$: 0.25.
- In between it goes **below zero**: roughly 150 to 450 slots apart, and again past about 650.

Near slots look alike, far slots less so. But "less" is not steady: the curve swings up and down. Next: what a negative cosine means.

Why the similarity goes below zero



gap 1: 0.997
almost the same direction



gap 100: 0.47
about 60 degrees apart



a few hundred apart: below 0
pointing more away than toward

- A cosine is an angle: $+1$ same direction, 0 perpendicular, -1 opposite. Below zero, the two rows point more **away** from each other than toward.
- View 2: each row is mostly "where its slot sits on the waves". Those waves take a few hundred slots per swing. Two slots half a swing apart sit on opposite sides, one near a crest, the other near a trough: their wave parts have opposite signs, so the rows point roughly opposite ways.
- A full swing apart they are back in step, and the curve climbs again.

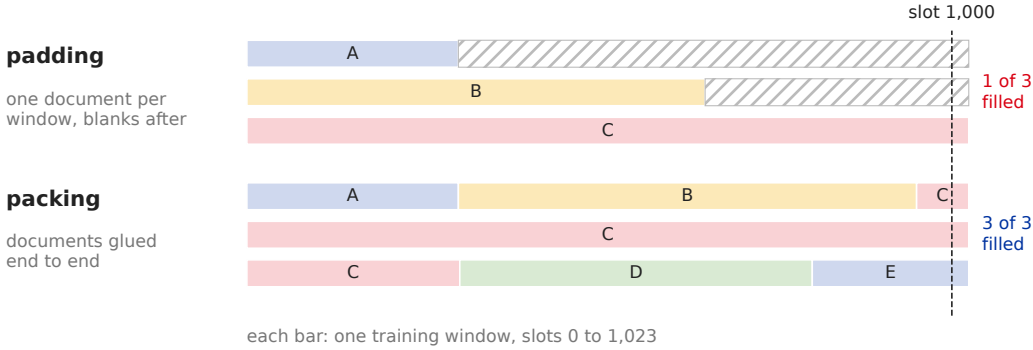
Order was learned from scratch, smooth and wave-like. Remember that for option 2: its formula dips below zero for exactly this reason.

Learned positions: the bill

- **A hard length limit.** The table has as many rows as the longest training text, and no row exists beyond it.
- **Nothing carries over.** Slot 2,000 cannot be guessed from slots 0 to 1,023: a new row needs training data at that length.
- **Absolute, not relative.** "Three slots back" has to be learned separately for slot 10, slot 500 and slot 1,000. Nothing in the setup says they are the same idea.

The waves on the last slides suggest a cheaper route: if smooth waves are what the model wants, write them down instead of learning them.

Are the last slots trained less often?



It depends on how the training text is cut into 1,024-token windows.

- **Padding**: one document per window, blanks after it. Then **yes**: slot 1,000 only trains on documents longer than 1,000 tokens, and most are shorter.
- **Packing**: documents glued end to end, cut into full windows. Then **no**: every slot is filled in every window, so slot 1,000 trains as often as slot 10.

Two real recipes

- **BERT (2018)** made the last slots rare on purpose: 90% of its training steps used 128-token windows, the last 10% used 512, "to learn the positional embeddings". Rows 128 to 511 of its position table trained in a tenth of the steps. Long windows cost more (attention compares every pair), so it saved them for the end.
- **GPT-3 (2020)** packs: it "always" trains on full 2,048-token windows, documents glued together with an end-of-text token between them. Every slot gets its share.

With packing, the last slots are fine. What stays rare is a long **gap** inside one document. More on that in the last section.

Devlin et al. (2018), BERT, appendix A.2. Brown et al. (2020), GPT-3, section 2.3.

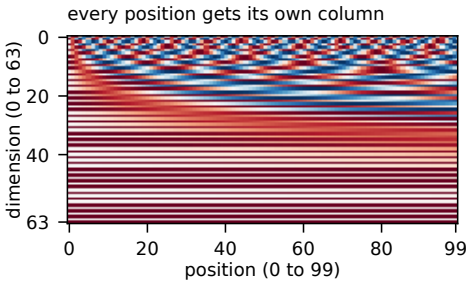
Option 2: write the positions down

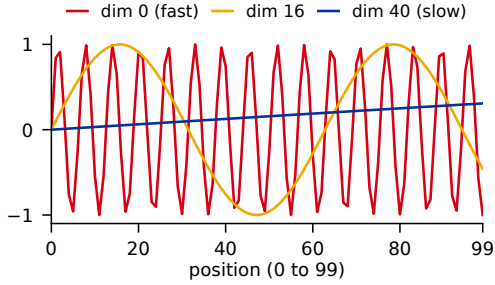
No training, and a value for every slot.

A clock with many hands

The original Transformer (Vaswani et al., 2017) fixes P with a formula. For slot *pos* and dimension pair i (of d dimensions):

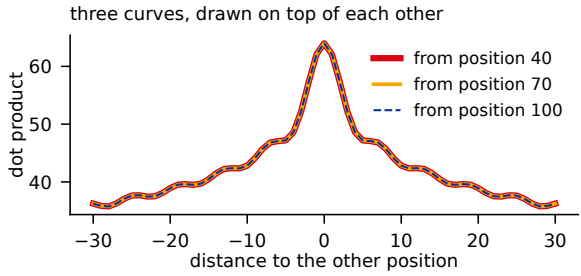
$$P_{pos,\,2i} = \sin\Big(\frac{pos}{10000^{2i/d}}\Big) \qquad P_{pos,\,2i+1} = \cos\Big(\frac{pos}{10000^{2i/d}}\Big)$$





Each pair of dimensions is a clock hand turning at its own speed. Fast hands tell neighbours apart; slow hands tell far-apart slots apart. Together they give every slot a unique reading ($d = 64$ here, $d = 128$ in the next plot).

Similarity depends on the gap only



The dot product of two slots' vectors, starting from slots 40, 70 and 100: the three curves are **identical** (largest difference below 10^{-9}). Only the distance matters, not where you are.

So for the bare waves, "three slots back" looks the same at slot 10 and at slot 1,000. Inside a real model it gets messier (option 3 says why).

The formula has a value for every slot, so it seems to promise texts longer than any seen in training. In practice it does not deliver: trained on 512 or 1,024 tokens, such models get worse soon after the training length (Press et al., 2022). The model never learned to use the unseen readings.

Why compare two slots? One clock hand, by hand

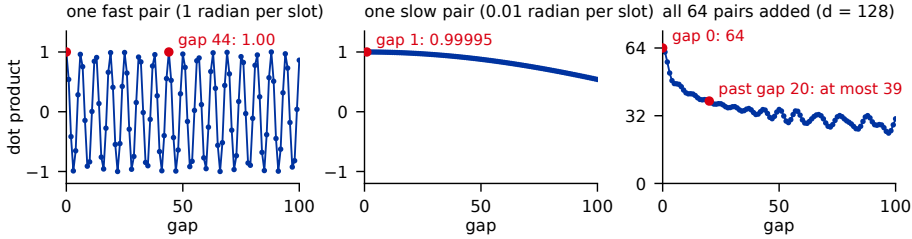
- The next layer compares vectors with dot products (the shuffle toy). So what position contributes is how two slots' vectors compare: $P_a \cdot P_b$. We want that to depend on how **far apart** a and b are, nothing else.
- Take a single pair of dimensions turning 30° per slot. Slot a holds $(\sin 30a^\circ,\ \cos 30a^\circ)$: an arrow of length 1, turned $30a^\circ$.

slots	angles	vectors	dot product
1 and 3	30°, 90°	$(0.5,\ 0.87) \cdot (1,\ 0)$	$0.5 + 0 = \mathbf{0.5}$
6 and 8	180°, 240°	$(0,\ -1) \cdot (-0.87,\ -0.5)$	$0 + 0.5 = \mathbf{0.5}$

Both pairs are 2 slots apart, so their arrows are 60° apart, and $\cos 60^\circ = 0.5$.

For arrows of length 1, the dot product is the cosine of the angle between them. In symbols, with speed ω: $\sin(\omega a)\sin(\omega b) + \cos(\omega a)\cos(\omega b) = \cos\big(\omega(a-b)\big)$. Only the gap is left.

Why one hand is not enough



- A **fast** hand alone repeats: in 44 slots it turns 7 times round, so gap 44 scores 1.00, like gap 0. It cannot tell near from far.
- A **slow** hand alone barely moves: gap 1 scores 0.99995. It cannot tell neighbours apart.
- All 64 hands added: only gap 0 reaches 64. Gap 1 still scores 62 (neighbours alike), and gaps 20 to 100 stay at 39 or below.

Like a clock: the seconds hand tells neighbouring moments apart, the hour hand far-apart ones. That is why there are many pairs.

All the hands together: only the gap is left

Pair i turns at its own speed ω_i and adds $\cos\big(\omega_i(a-b)\big)$ to the dot product (two slides back). Add up all $d/2$ pairs:

$$P_a \cdot P_b \;=\; \sum_i \cos\big(\omega_i\,(a-b)\big), \qquad \omega_i = \frac{1}{10000^{2i/d}}$$

- Only $a-b$ is left: the address cancels. That is why the three curves were one.
- At gap 0 every term is $\cos 0 = 1$, so the dot product peaks at $d/2$: 64 in the plot, where $d = 128$.
- Further apart, the pairs turn at different speeds and drift out of step; their terms partly cancel. Those are the wobbles.

Hold on to $\cos(\omega(a-b))$: RoPE (option 3) is built on exactly this identity.

Every model here adds the position vector to the token vector. Why not put them side by side?

Adding mixes two kinds of information into the same 768 numbers. Surely that blurs the token?

Commit to an answer.

By hand: adding can be concatenation in disguise

A toy with 4 numbers per vector. Tokens use the first two, slots the last two:

$$\text{dog} = (3,1,0,0) \quad \text{cat} = (1,3,0,0) \qquad \text{slot 1} = (0,0,1,0) \quad \text{slot 4} = (0,0,0,1)$$

$$\text{dog in slot 4} = (3,1,0,0) + (0,0,0,1) = (3,1,0,1)$$

- The first two numbers still say "dog": the dot product with dog is $3 \cdot 3 + 1 \cdot 1 = 10$, with cat only $3 \cdot 1 + 1 \cdot 3 = 6$.
- The last two numbers say "slot 4". Nothing was blurred.
- Putting them side by side, $(3,1\,|\,0,1)$, is the same thing, except that every later layer must be wider to accept the longer vector.

If tokens and slots live in **different directions**, adding them loses nothing. The question is whether a real model keeps them apart.

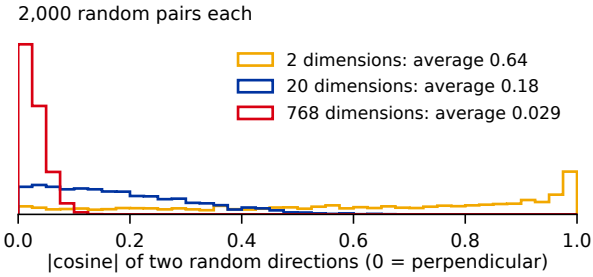
Side by side has a price

Three ways to give a 768-number token a 768-number position:

- **Side by side, 1,536 numbers.** Every weight matrix in every layer grows on both sides, so each layer gets about 4 times the weights: GPT-2's 12 layers would go from 85M to 340M parameters.
- **Side by side inside 768**, say 640 for the token and 128 for the slot. No growth, but every token starts with 128 fewer numbers of its own.
- **Add.** The token keeps all 768, the slot gets all 768, nothing grows. The catch: both share the same numbers.

And side by side is less different than it looks. The first thing a layer does is multiply by a matrix. Cut the matrix into a token part and a slot part: $W \begin{pmatrix} e \\ p \end{pmatrix} = W_1 e + W_2 p$. Side by side, then a matrix, **is** adding, just of two transformed vectors.

First: random directions in 768 dimensions

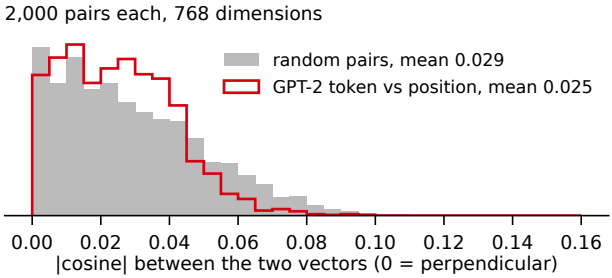


Draw two directions at random and take $|\cos|$ of the angle between them: 0 means perpendicular, 1 means the same line.

In 2 dimensions it can be anything: average 0.64. In 768 dimensions two random directions are almost always nearly perpendicular: average 0.029.

So 768 dimensions leave room for many directions that barely overlap. If token rows and slot rows sit in such directions, adding them blurs almost nothing. Does GPT-2 do that? Two tests on its real tables.

Test 1: do GPT-2's token rows and slot rows overlap?



Red: 2,000 random pairs (one token row, one slot row) from GPT-2's two tables, $|\cos|$ of each pair.

Grey: 2,000 pairs of random directions, as on the last slide.

Average $|\cos|$: **0.025** for GPT-2's pairs, 0.029 for random ones. Token rows and slot rows overlap **no more than random directions do**: they ended up nearly perpendicular.

Test 1 says the directions barely overlap. Test 2 checks what that buys: after adding, is the token still readable?

Test 2: can we still read the token after adding?

1. Pick a random token and a random slot. Add their rows: the input the model really gets.
2. Ask which of the 50,257 token rows points most the same way as that sum.
3. Count it as right if that is the token we started from.

The 4-number toy did the same by hand: $(3, 1, 0, 1)$ still scored highest with dog (10, against 6 for cat).

On GPT-2: right **1,998 times out of 2,000**. The 2 misses: a stray byte and " RandomRedditor", a glitch token. The barely-trained rows again. Even slot 0, whose row is about three times longer than the rest, keeps 1,997 of 2,000.

The answer to the predict slide: adding does not blur the token. 768 dimensions leave room for directions that barely overlap, so adding costs nothing and keeps both messages readable.

Option 3: rotate, do not add

What a comparison needs is the gap, not the address.

What attention needs from position

The next layer compares pairs of tokens (LLM-4). From each token's vector it makes two new ones: a **query** ("what am I looking for?") and a **key** ("what do I offer?"). The score of a pair is query · key, the dot product from the toy.

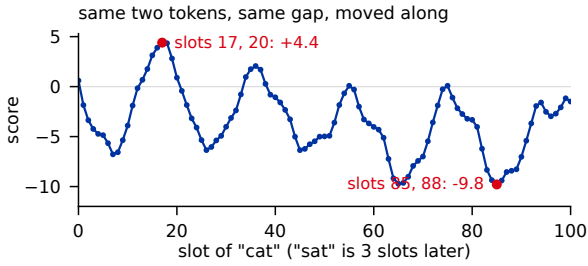
The same five words, once at the start of a text and once deep inside it:

slots	0	1	2	3	4
tokens	The	**cat**	then	quietly	**sat**
slots	40	41	42	43	44
tokens	the	**cat**	then	quietly	**sat**

"sat" looks back for whoever sat: "cat", 3 slots earlier, both times. Same words, same gap, so the score should be the same. Where the pair sits in the text should not matter.

Predict: with a position vector *added* to each token (options 1 and 2), is the score of "sat" looking at "cat" the same at slots 4 and 1 as at 44 and 41?

A toy says no: the address leaks in



A toy layer: 16-number vectors for "cat" and "sat", random query and key matrices, sinusoid positions added to both.

Slide the pair along the text, always 3 apart, and score it at every place.

Same two tokens, same gap, and the score swings from -9.8 to $+4.4$ only because of **where** the pair sits. The model would have to learn to undo that.

Toy numbers (random vectors, seed 509), not a trained model: the point is that nothing stops the address from leaking in. Plot: `py_src/embeddings_position_figs.py`.

Why the address leaks in

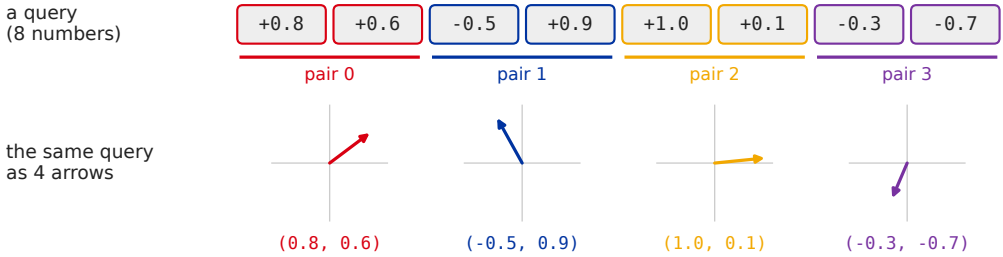
Write $x = \text{token} + \text{position}$, and let $M = W_q^\top W_k$ stand for the two learned matrices. The score of "sat" in slot n looking at "cat" in slot m, multiplied out:

$$(e_{\text{sat}} + P_n)^\top M\,(e_{\text{cat}} + P_m) \;=\; \underbrace{e_{\text{sat}}^\top M e_{\text{cat}}}_{\text{content}} + \underbrace{e_{\text{sat}}^\top M P_m}_{\text{slot } m \text{ alone}} + \underbrace{P_n^\top M e_{\text{cat}}}_{\text{slot } n \text{ alone}} + \underbrace{P_n^\top M P_m}_{\text{both slots}}$$

- The two middle terms each depend on **one** slot: the address, not the gap.
- Even the last term depends on the gap alone only when M is the identity matrix (bare waves: the $\cos(\omega(a-b))$ result a few slides back), not after learned matrices.

Better: build the comparison so that **only the gap can affect it**. Rotary Position Embedding (RoPE; Su et al., 2021): instead of adding a position vector, **rotate** the query and the key by an angle proportional to their slot.

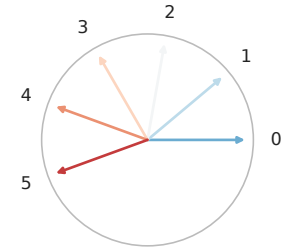
RoPE, step 1: cut the vector into pairs



- Numbers 1 and 2 form pair 0, numbers 3 and 4 form pair 1, and so on. Each pair is a point in 2-D: draw it as an arrow from the centre.
- A real query or key in a Llama-style model has 128 numbers (each head, LLM-5, works on its own slice of the token vector): **64 arrows**. Nothing is added or lost.

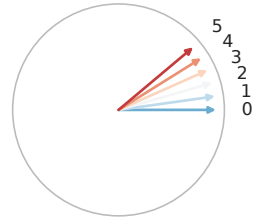
RoFormer (2021) pairs neighbouring numbers, as here. Llama's code pairs number i with number $i + 64$ instead: the same turns, different bookkeeping.

RoPE, step 2: turn each arrow by its slot



a fast pair: 40 degrees per slot

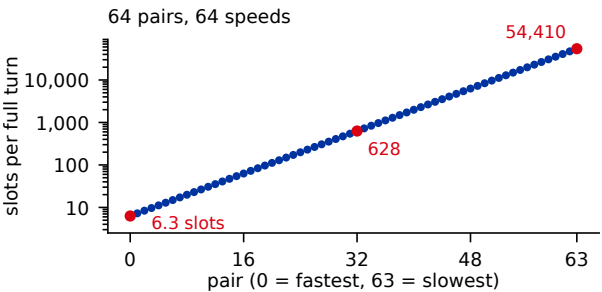
same token,
slots 0 to 5



a slow pair: 8 degrees per slot

- One pair of the same token, in slots 0 to 5. In slot m, a pair with speed θ is turned by $m \times \theta$: the fast pair $40°$ per slot, the slow pair $8°$.
- Turning (x, y) by an angle φ gives $(x\cos\varphi - y\sin\varphi,\ x\sin\varphi + y\cos\varphi)$. Example: $(1, 0)$ turned by $90°$ is $(0, 1)$.
- The arrow's length never changes: only its direction carries the slot. That is why we call it a clock hand.

RoPE, step 3: every pair has its own speed

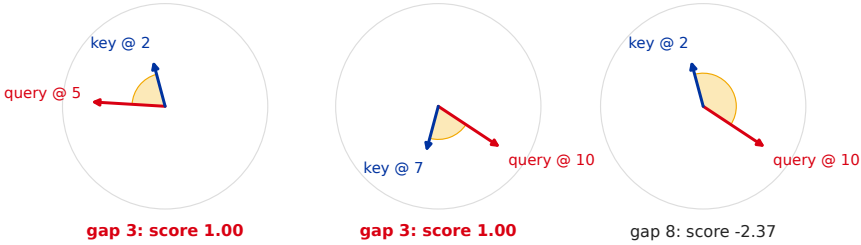


Pair i turns $\theta_i = 1/10000^{2i/d}$ radians per slot: the sinusoids' speeds. With 128 numbers ($d = 128$), 64 pairs:

- pair 0: 1 radian per slot, a full turn every 6.3 slots;
- pair 32: 0.01, every 628;
- pair 63: 0.000115, every 54,410.

Fast hands tell neighbours apart, slow hands tell far-apart slots apart: the clock of option 2, but now turning the query and the key themselves.

Why only the gap survives: the picture



1. A dot product depends on two lengths and the angle between the arrows: $\mathbf{q} \cdot \mathbf{k} = |\mathbf{q}|\,|\mathbf{k}| \cos(\text{angle between})$.
2. Turn the query by $m\theta$ and the key by $n\theta$. The lengths do not change. The angle between them is the unturned angle plus $(m - n)\,\theta$.
3. So the score depends on the content and the **gap** $m - n$ only. Left and middle: gap 3 at different slots, the same score 1.00. Right: gap 8, another score.

Why only the gap survives: the algebra

Write a turn by an angle φ as a matrix, $R(\varphi) = \begin{pmatrix} \cos\varphi & -\sin\varphi \\ \sin\varphi & \cos\varphi \end{pmatrix}$, and let $R_m = R(m\theta)$ be the turn in slot m.

1. A dot product is a transpose times a vector: $(R_m\mathbf{q}) \cdot (R_n\mathbf{k}) = \mathbf{q}^\top R_m^\top R_n\,\mathbf{k}$.
2. Transposing a turn undoes it: $R_m^\top = R_{-m}$. (Transposing swaps the two sines, and $\cos(-x) = \cos x$, $\sin(-x) = -\sin x$.)
3. Two turns add up: $R_{-m}R_n = R_{n-m}$. Back by m, then forward by n, is one turn by $n - m$.

$$(R_m\mathbf{q}) \cdot (R_n\mathbf{k}) \;=\; \mathbf{q}^\top R_{n-m}\,\mathbf{k}$$

The slots m and n survive only as $n - m$: the same gap as $m - n$ in the picture, counted from the key's side.

A real query has 64 pairs, each with its own θ_i. Its score is the **sum** of the 64 pair scores, and each of them depends only on the gap.

By hand: same gap, same score

$\mathbf{q} = (2, 1)$, $\mathbf{k} = (1, 1)$, $\theta = 30°$ per slot (a toy speed). Unrotated, $\mathbf{q} \cdot \mathbf{k} = 2 + 1 = 3$. Turning (x, y) by an angle φ gives $(x\cos\varphi - y\sin\varphi,\ x\sin\varphi + y\cos\varphi)$.

slots	**query turned by**	**key turned by**	**score**
q in 5, k in 2	150°: $(-2.23,\ 0.13)$	60°: $(-0.37,\ 1.37)$	$0.82 + 0.18 = \mathbf{1.00}$
q in 10, k in 7	300°: $(1.87,\ -1.23)$	210°: $(-0.37,\ -1.37)$	$-0.68 + 1.68 = \mathbf{1.00}$
q in 10, k in 2	300°: $(1.87,\ -1.23)$	60°: $(-0.37,\ 1.37)$	-2.37

Gap 3 gives 1.00 at slots 5/2 and at 10/7. Gap 8 gives something else. Swap the roles (q in 2, k in 5: gap -3) and it is -1.00: the gap is signed, so before and after differ. Check one row yourself: $\cos 150° = -0.866$, $\sin 150° = 0.5$, so the query becomes $(2 \cdot (-0.866) - 1 \cdot 0.5,\ 2 \cdot 0.5 + 1 \cdot (-0.866)) = (-2.23,\ 0.13)$.

The model never sees the slot numbers 5, 2, 10 or 7. It only ever feels the gap.

By hand: four numbers, two speeds

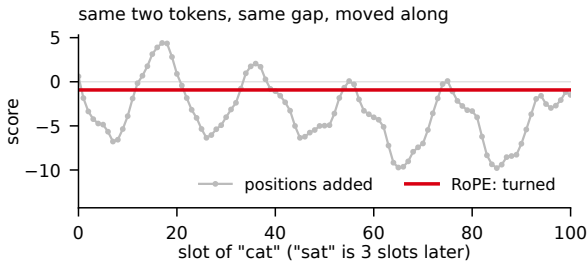
Two pairs: $\mathbf{q} = \mathbf{k} = (1, 0 \,|\, 1, 0)$. The fast pair turns 90° per slot, the slow pair 30°. Each pair's arrows have length 1, so its score is the cosine of the angle between them, and the total score is the sum of the two:

gap	**fast pair** (90°)	**slow pair** (30°)	**score**
0	$\cos 0^\circ = 1$	$\cos 0^\circ = 1$	**2**
1	$\cos 90^\circ = 0$	$\cos 30^\circ = 0.87$	0.87
2	$\cos 180^\circ = -1$	$\cos 60^\circ = 0.5$	-0.5
4	$\cos 360^\circ = 1$	$\cos 120^\circ = -0.5$	0.5

Check gap 4 by turning both, $\mathbf{q}$ in slot 5 and $\mathbf{k}$ in slot 1. Fast pair: $(0, 1) \cdot (0, 1) = 1$. Slow pair: $(-0.866, 0.5) \cdot (0.866, 0.5) = -0.75 + 0.25 = -0.5$. Total 0.5, and slots 9 and 5 give 0.5 again.

The fast pair alone cannot tell gap 4 from gap 0: one full turn looks like no turn. The slow pair can. That is why RoPE mixes many speeds, from 1 radian per slot down to 0.000115.

Back to cat and sat

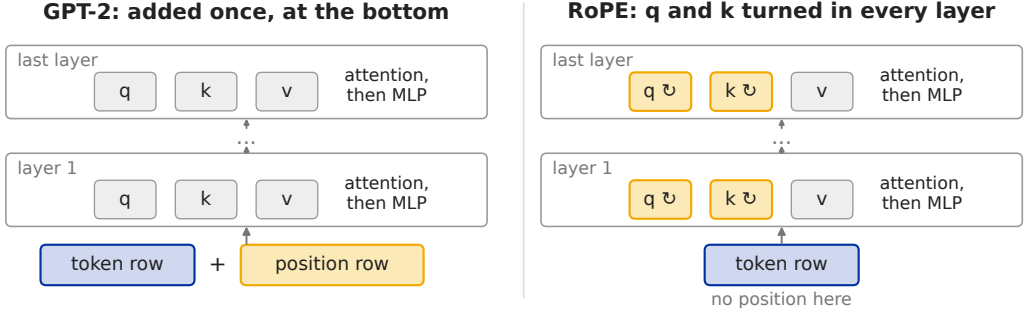


The toy from "the address leaks in": same vectors, same matrices. Only one change: instead of adding positions, turn the query by its slot and the key by its slot.

Red: the score is now **one number** wherever the pair sits (equal to 10^{-9}).

The address no longer leaks in: only the gap is left. That is the whole point of turning instead of adding.

Where the rotation happens



- GPT-2 (left): the position row is added **once**, at the bottom, and carried up through all 12 layers.
- RoPE (right): no position at the input. In every layer, each token's query and key are turned by its slot right before their dot product. The value v, the content passed along, is not turned: position only decides who looks at whom.

With RoPE, the map's Position box acts inside attention, in every layer.

Who uses it

- RoFormer (2021) reported modest gains. Adoption was anything but modest: GPT-J (2021) and GPT-NeoX (2022), PaLM (2022), then LLaMA (2023), and after LLaMA nearly every open model.
- Every 2024-2026 model in the table at the end of today uses RoPE in some form.
- Two reasons it won: the score depends only on the gap even after the learned matrices (added sinusoids lose that: the address leaks in), and, like sinusoids, it adds no parameters at all - a rotation has nothing to learn.

One thing RoPE does not fix by itself: texts longer than the training ones. That is the last section.

Three options, side by side

	learned table (GPT-2)	**sinusoids** (2017)	**RoPE** (2021)
what it does	adds a trained row per slot	adds a fixed wave pattern	rotates query and key inside attention
parameters	one row per slot (786,432 numbers in GPT-2)	none	none
"3 slots apart"	learned separately at every slot	the same pattern everywhere; the model must dig it out	the score sees only the gap
past the training length	impossible: no row exists	allowed, but quality drops	allowed, but quality drops; can be stretched (next section)

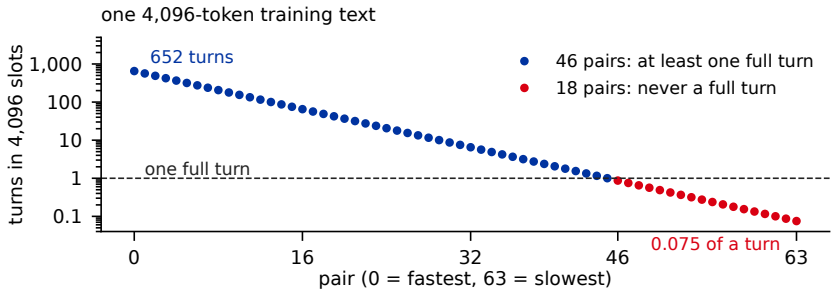
Each option fixes something in the column before it: sinusoids drop the trained rows and make the gap the same everywhere; RoPE makes the gap the **only** thing a comparison can see. None of them, by itself, makes a model good at lengths it never trained on.

Longer and longer

GPT-2 stopped at 1,024 tokens. Models now take a million - and do not read all of it equally well.

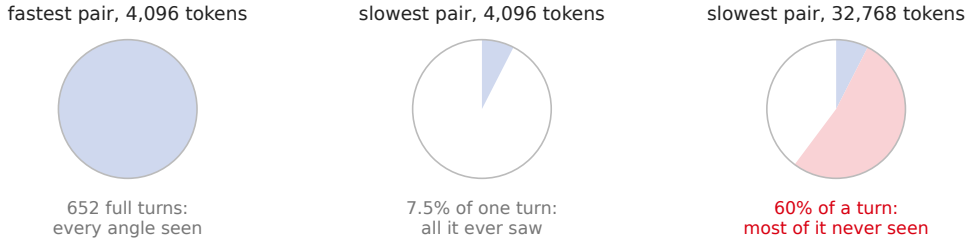
Which hands saw every angle in training?

Reminder. A **hand** is one pair of numbers in a query or key, drawn as an arrow. Its **speed** is how far it turns per slot. 64 hands, from 1 radian per slot (a full turn every 6.3 slots) down to 0.000115 (a full turn every 54,410 slots).



A RoPE (2021) model trained on 4,096-token texts, base 10,000, 128-number queries and keys (Llama 2's settings, 2023). In one training text the 46 fastest hands go round at least once: they have shown the model every angle. The 18 slowest never complete a turn.

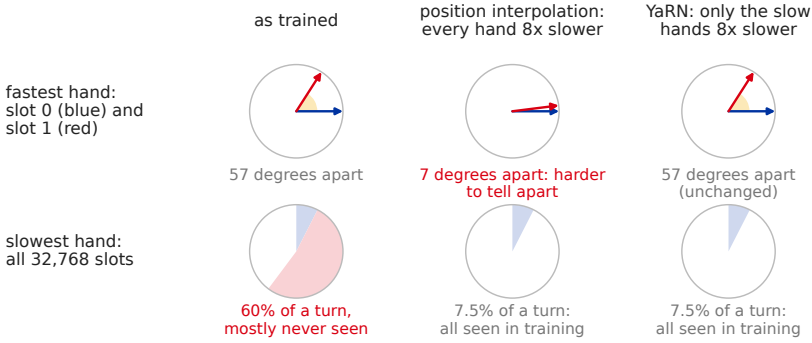
Why RoPE breaks past its training length



- Fastest hand, pair 0 (left): 1 radian per slot, about 650 full turns in 4,096 slots.
- Slowest hand, pair 63 (middle): $4{,}096 \times 0.000115 = 0.47$ radians, only **7.5%** of a circle. That thin slice is all the model ever saw of this hand.
- Feed it 32,768 tokens (right): the same hand reaches 60% of its circle. The weights were fitted to the seen slice only, so in the red part the scores are a guess.

Press et al. (2022) measured it: RoPE models hold up for 100-200 tokens past their training length, then get worse. Sinusoids fail sooner.

Stretch the clock



- Trained on 4,096 slots, asked for 32,768. **Position interpolation** (Chen et al., 2023): every hand 8 times slower. The slow hand stays in its known slice; neighbours end up 7 degrees apart.
- **YaRN** (Yet another RoPE extensioN; Peng et al., 2024): only the slow hands get slower. Fast hands keep their 57 degrees; hands in between get a mix.

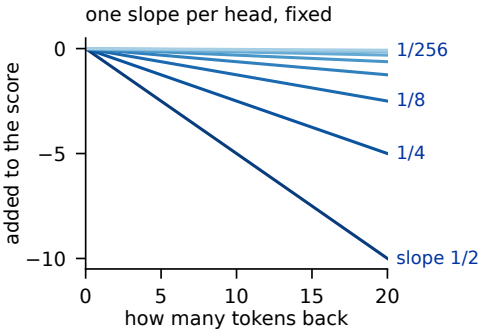
Stretching in practice

- Position interpolation reads slot 32,000 as 4,000, an angle the model knows. The price is the top row of the last picture: neighbours are harder to tell apart.
- YaRN keeps the fast hands, so neighbours stay sharp, and fixes only the hands that would otherwise reach unseen angles (the red part two slides back).
- Either way, then a short extra training run on long documents, so the model gets used to the new speeds.

In 2025 configs: Qwen3-8B (2025), native length 32,768 tokens, reaches 131,072 with YaRN. gpt-oss-20b (2025) was stretched from 4,096 to 131,072.

Qwen3-8B model card; gpt-oss-20b config.json (`rope_scaling`), checked 4 Oct 2026.

Another answer: a distance penalty (ALiBi)

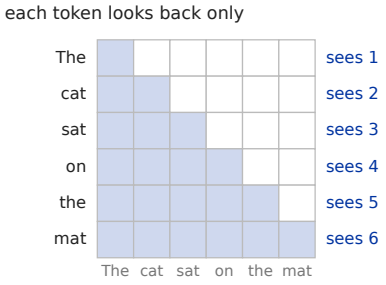


Attention with Linear Biases (ALiBi; Press et al., 2022): no position vectors at all. Each comparison score gets a penalty: slope $\times$ distance.

A model runs several comparisons side by side, called heads (LLM-5). Each head gets its own slope, fixed before training: $1/2, 1/4, \ldots, 1/256$. Steep heads look near, flat heads look far.

Near tokens win unless a far one is much more relevant. The paper's title says what it was built for: "Train Short, Test Long". BLOOM (2022) used it.

Or no position at all (NoPE)



No Positional Encoding (NoPE): nothing. A model that generates one token at a time (LLM-2) lets each token look only at the ones before it (LLM-4).

Such a model still works out positions on its own (Haviv et al., 2022): token 3 sees 3 tokens, token 300 sees 300. How the count shows: an average over 3 vectors wobbles more than an average over 300, and later layers can read that spread (Chi et al., 2023).

Llama 4 (2025) mixes RoPE and NoPE: some layers rotate, some have no position at all. Meta reports up to 10 million tokens of context for Llama 4 Scout.

Position is still an open design question. The 2017 answer did not last, and the 2021 answer keeps being patched.

What models use, 2017-2026

model	position	context (tokens)
Transformer (2017)	sinusoids	-
BERT (2018)	learned table	512
GPT-2 (2019)	learned table	1,024
T5 (2019)	a learned bias per distance	512 in training
BLOOM (2022)	ALiBi	2,048
Llama 3.1 (2024)	RoPE, base 500,000, stretched from 8,192	131,072
DeepSeek-V3 (2024)	RoPE + YaRN (from 4,096)	163,840
Qwen3-8B (2025)	RoPE, base 1,000,000	32,768; 131,072 with YaRN
gpt-oss-20b (2025)	RoPE + YaRN (from 4,096)	131,072
Llama 4 Scout (2025)	RoPE layers + NoPE layers	10,000,000
Gemma 4 E2B (2026)	RoPE, two bases: 10,000 and 1,000,000	131,072

Base: the 10,000 in the speed formula (RoPE step 3); a bigger base slows the slow hands. Gemma: layers that look only nearby use 10,000, layers that see the whole text 1,000,000. Closed models, Oct 2026: Gemini 4 Argon 1,048,576, Claude Opus 5.5 and Fable 5.1 1M, GPT-6 Astra 1,050,000; position scheme not published.

config.json and model cards on Hugging Face, Meta's Llama 4 announcement, checked 4 Oct 2026; Llama 3.1's config and the vendors' model pages, checked 27 Oct 2026.

Why 131,072 and 163,840?

- $131{,}072 = 2^{17} = 128 \times 1{,}024$: the "128K" on the box, counting K as 1,024. Llama 3.1 (2024) and Gemma 4 E2B (2026) list it as is. Llama 3.1's stretch factor (8, from 8,192) would give only 65,536: it also trained on texts up to 128K at the end (last section).
- Stretched models multiply: the length they trained at, times the stretch factor in their config.
  - DeepSeek-V3 (2024): $4{,}096 \times 40 = \mathbf{163{,}840}$
  - gpt-oss-20b (2025): $4{,}096 \times 32 = 131{,}072$
  - Qwen3-8B (2025): $32{,}768 \times 4 = 131{,}072$
- Why 40 and not 32? The config does not say.

Powers of two are a habit and a convenience, not magic: attention code on GPUs usually works through the text in blocks of 64 or 128 tokens, and $163{,}840 = 1{,}280 \times 128$ divides just as evenly.

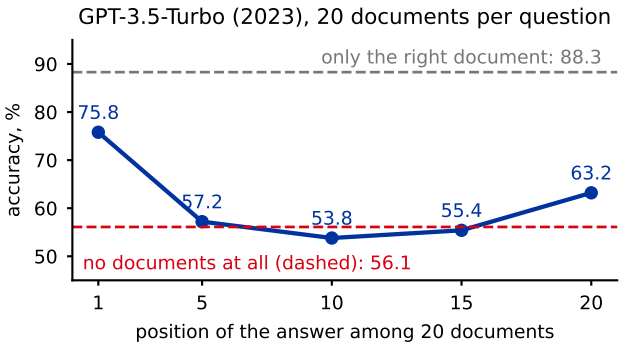
`max_position_embeddings` and `rope_scaling` in each config.json on Hugging Face, checked 28 Oct 2026; Qwen3-8B model card.

A question, and 20 documents in the prompt.
One of them holds the answer.

Does it matter whether that document comes first, in the middle, or last?

Commit to an answer.

Lost in the middle



The question comes after the documents. A U-shape: the answer is found best at the **start**, then at the **end**.

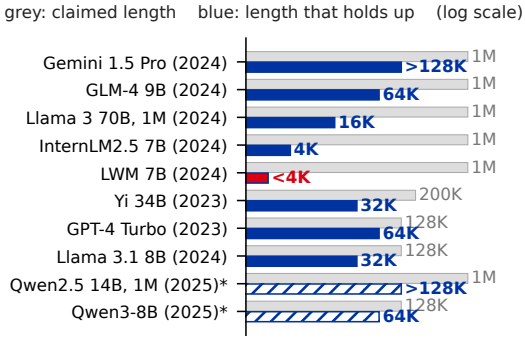
In the middle, GPT-3.5 Turbo (2023) does **worse than with no documents at all** (53.8% vs 56.1%): there, the extra text actively hurts.

The paper measures the effect; it does not settle the cause.

A window that *fits* 20 documents is not a model that *uses* all 20. In a long prompt, put what matters first or last.

Liu et al. (2024), "Lost in the Middle: How Language Models Use Long Contexts", TACL; NaturalQuestions (2019), 20 Wikipedia passages per question; model gpt-3.5-turbo-0613. Plot: `py_src/long_context_figs.py`.

Claimed length vs used length



* hatched: run by the model's makers, not the RULER team

The classic test hides one sentence, a "needle", in a long text and asks for it back. Most models now pass it.

RULER (Hsieh et al., 2024): 13 harder tasks, such as several needles at once or tracing a variable. "Holds up" = still beats what Llama 2 7B (2023) scores at 4K tokens (85.6%). Tested up to 128K.

RULER's first 17 models (2023-2024) all claimed 32K or more; "only half" held up at 32K. Three of the five 2024 1M models here fail before 32K. The hatched 2025 rows do better, by their makers' runs.

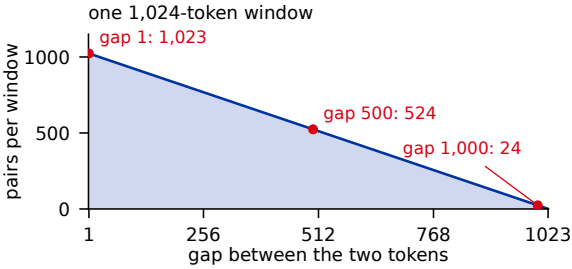
RULER leaderboard (github.com/NVIDIA/RULER), read 6 Nov 2026; "Llama 3 70B, 1M" is Gradient's.

Context rot, and what to take away

- **Context rot** (Chroma, July 2025): 18 models, simple tasks, longer and longer inputs. The answers get steadily less reliable as the input grows.
- Oddly, a shuffled haystack was easier than a coherent one.
- Same picture as RULER and lost in the middle: the window is a promise about what fits, not about what gets used.

Position encoding decides how far a model *can* read. Training decides how well it reads that far. Test the length you need; do not trust the number on the box.

Long gaps are rare in training



Back to the question from option 1. In one 1,024-token window, two tokens 1 apart occur 1,023 times, 1,000 apart only 24 times: a window of L tokens holds $L - k$ pairs k apart.

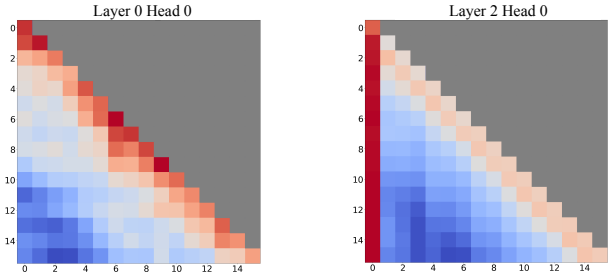
And a long gap only teaches something if both tokens sit in the **same** document. Most documents are short; Llama 3 (2024) even blocks attention across document borders in a packed window.

So models train short and stretch late. Llama 3 (2024): 8K-token windows for the main run of 15.6T tokens, then six stages up to 128K, about 800B tokens: roughly 5% of the training.

Far-apart pairs get the least practice. That is one likely reason the used length trails the claimed one.

Grattafiori et al. (2024), "The Llama 3 Herd of Models": the document mask, and long-context pre-training of the 405B model.

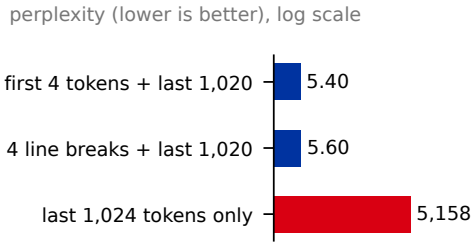
The first slots are special: attention sinks



Xiao et al. (2024), Figure 2: Llama 2 7B (2023), scores before softmax, averaged over 256 texts of 16 tokens. Row = the token looking, column = the token looked at; red = high, grey = not allowed (later tokens). Layers 9, 16, 23 and 31 in the paper look like layer 2.

- Layer 0 (left) looks at nearby tokens. From layer 2 on (right), every token's highest score, and so after softmax its largest weight, goes to **token 0**, whatever it says.
- Why: softmax must hand out weights that sum to 1. When nothing is relevant, the model parks the spare attention on a token every later position can see: the first one.

Drop the sink and the model breaks



Llama 2 13B (2023) on a 65,000-token book, reading with a window of 1,024 tokens. Perplexity: how surprised the model is by the text, lower is better.

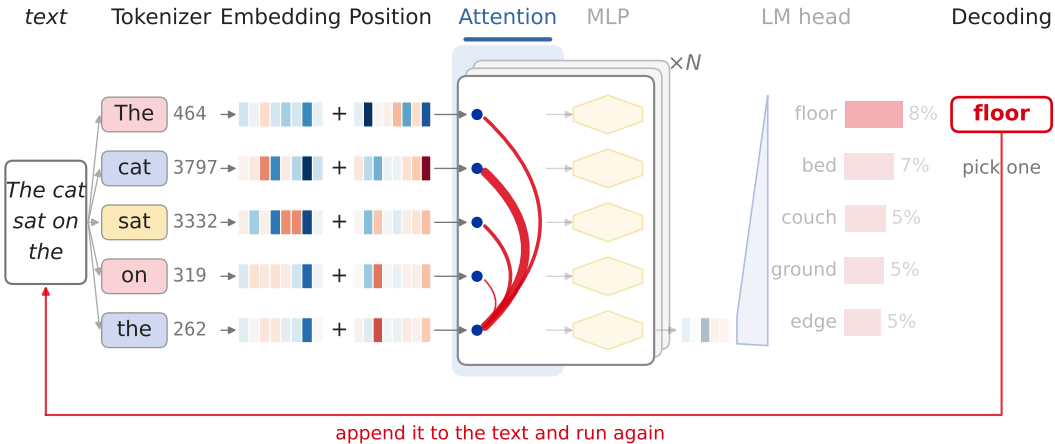
- Keep only the last 1,024 tokens and it explodes to 5,158. Keep the first 4 as well and it is fine: 5.40. Swap those 4 for line breaks: 5.60. Their position matters, not their meaning.
- Their fix, StreamingLLM: always keep the first 4 tokens plus a sliding window of recent ones. Stable over 4 million tokens.

Slot 0 is special in GPT-2's own position table too: its row is about three times longer than the median slot's (9.88 vs 3.37), an easy mark for a head to find. LLM-5 measures the sinks in GPT-2's heads.

Recap

- An ID picks a **row of a learned table** (one-hot times a matrix). The rows start as noise and learn how tokens are used (same next words, same nudges): days near days, opposites near each other, glitch tokens in a pile. Later layers make each occurrence different.
- A lookup ignores order: both dog sentences arrive as the same five vectors.
- So each vector also gets its **slot**: a learned row (GPT-2, hard limit 1,024), a formula (sinusoids, 2017), or a rotation inside every attention layer (RoPE, 2021: only the gap counts).
- Adding works because in 768 dimensions token and position directions barely overlap.
- Longer texts: stretch the slow clock hands (YaRN, 2024), or penalise distance (ALiBi, 2022), or drop position in some layers (NoPE, 2022).
- Fitting is not using: answers in the middle of a long prompt get lost, claimed lengths outrun used ones, and the first tokens soak up spare attention (sinks).

Next on the map



Next (LLM-4): the vectors talk to each other. (The map is GPT-2, which adds positions.)

Sources & further reading (1/2)

- **Watch:** 3Blue1Brown (2024), Transformers, the tech behind LLMs | Deep Learning Chapter 5; Jia-Bin Huang (2024), How Rotary Position Embedding Supercharges Modern LLMs [RoPE]. **Play:** projector.tensorflow.org.
- Vaswani et al. (2017), "Attention Is All You Need", NeurIPS (sinusoids).
- Su et al. (2021), "RoFormer: Enhanced Transformer with Rotary Position Embedding", arXiv; Neurocomputing (2024).
- Press, Smith & Lewis (2022), "Train Short, Test Long: Attention with Linear Biases Enables Input Length Extrapolation", ICLR.
- Haviv et al. (2022), "Transformer Language Models without Positional Encodings Still Learn Positional Information", Findings of EMNLP.
- Chen et al. (2023), "Extending Context Window of Large Language Models via Positional Interpolation", arXiv; Peng et al. (2024), "YaRN", ICLR.
- Meta (April 2025), "The Llama 4 herd"; Grattafiori et al. (2024), "The Llama 3 Herd of Models", arXiv.

Every GPT-2 number: `py_src/embeddings_position_figs.py` (GPT-2's weights read with safetensors, IDs from tiktoken), raw results in `results/embeddings_position_figs.json`.

Sources & further reading (2/2)

- Firth (1957), "A synopsis of linguistic theory, 1930-1955", *Studies in Linguistic Analysis*.
- Mikolov, Yih & Zweig (2013), "Linguistic Regularities in Continuous Space Word Representations", NAACL.
- Bolukbasi et al. (2016), "Man is to Computer Programmer as Woman is to Homemaker? Debiasing Word Embeddings", NeurIPS. Nissim, van Noord & van der Goot (2020), "Fair is Better than Sensational: Man is to Doctor as Woman is to Doctor", Computational Linguistics.
- Land & Bartolo (2024), "Fishing for Magikarp", EMNLP.
- Liu et al. (2024), "Lost in the Middle: How Language Models Use Long Contexts", TACL.
- Hsieh et al. (2024), "RULER: What's the Real Context Size of Your Long-Context Language Models?", COLM. Hong, Troynikov & Huber (2025), "Context Rot", Chroma technical report.
- Chi et al. (2023), "Latent Positional Information is in the Self-Attention Variance of Transformer Language Models Without Positional Embeddings", ACL.
- Xiao et al. (2024), "Efficient Streaming Language Models with Attention Sinks", ICLR (Figure 2 reproduced from the arXiv source).
- RULER leaderboard, github.com/NVIDIA/RULER (read 6 Nov 2026). Zhang et al. (2025), "Qwen3 Embedding", arXiv; OpenAI, embeddings guide.